

Final Project: Federated Learning & Its Attacks and Defenses

Version: 1

Step 1: Establish a Federated Learning System

- Establish a Federated Learning System
 - You can apply it for face recognition, for example,
 - [GitHub - mikemikezhu/federated-learning-facial-expression-recognition](https://github.com/mikemikezhu/federated-learning-facial-expression-recognition)
 - [GitHub - d3-ai/face-verification-demo: Face Verification Example with Flower / Federated Learning](https://github.com/d3-ai/face-verification-demo)
 - Also okay for the other domain data (e.g., text data)
 - Some libraries for Federated Learning
 - <https://www.tensorflow.org/federated>
 - [Top 7 Open-Source Frameworks for Federated Learning - www.apheris.com](https://www.apheris.com)
 - ...

Step 2: Gradient Leakage Attack in Federated Learning

- Apply Gradient Leakage Attack for the Federated Learning system (from Step 1): try to recover its original data of each client
- You can apply Gradient Leakage Attack from the list below: (but not limited in them)
 - Optimization-based attack method
 - Deep Leakage from Gradients : <https://proceedings.neurips.cc/paper/2019/file/60a6c4002cc7b29142def8871531281a-Paper.pdf>
 - iDLG: Improved Deep Leakage from Gradients: <https://arxiv.org/pdf/2001.02610>
 - Inverting Gradients - How easy is it to break privacy in federated learning? : <https://proceedings.neurips.cc/paper/2020/file/c4ede56bbd98819ae6112b20ac6bf145-Paper.pdf>
 - Analytics-based attack method
 - Privacy-Preserving Deep Learning: Revisited and Enhanced: <https://link.springer.com/content/pdf/10.1007/978-981-10-5421-1.pdf?pdf=button%20sticky>
 - Rethinking privacy preserving deep learning: How to evaluate and thwart privacy attacks: <https://arxiv.org/pdf/2006.11601>
 - R-gap: Recursive gradient attack on privacy: <https://arxiv.org/pdf/2010.07733>
- Most of above papers had shared their source code on Github or internet.
- Make the experiments to observe the results
 - Showcase successful instances of hacking or recovering original image data from gradients
 - Share your observations

Step 3: Defend against Gradient Leakage Attack, by Differential Privacy

- Try to defend against Gradient Leakage Attack (from Step 2)
- You can apply the Differential Privacy method from the below (but not limited in them)
 - Centralized DP: [M. Abadi, et al. "Deep learning with differential privacy." ACM SIGSAC conference on computer and communications security. 2016.](#)
 - Source code: [GitHub - lingyunhao/Deep-Learning-with-Differential-Privacy: An implementation of Deep Learning with Differential Privacy](#)
 - Some Reference:
 - [Deep Learning with Differential privacy](#)
 - [Deep Learning with Differential Privacy \(DP-SGD Explained\) \(mukulrathi.com\)](#)
 - [Introducing TensorFlow Privacy: Learning with Differential Privacy for Training Data — The TensorFlow Blog](#)
 - [GitHub - tensorflow/privacy: Library for training machine learning models with privacy for training data](#)
 - However, you need to additionally apply this method on Federated Learning environment
 - Distributed DP: [P. Kairouz, Z. Liu, and T. Steinke, "The distributed discrete gaussian mechanism for federated learning with secure aggregation." ICLR, 2021.](#)
 - Source code: https://github.com/google-research/federated/tree/master/distributed_dp
 - (Other resource) https://www.tensorflow.org/federated/api_docs/python/tff/learning/ddp_secure_aggregator
 - Others: [GitHub - git-disl/Fed-CDP: Gradient-Leakage Resilient Federated Learning](#)
- Make the experiments to observe the results
 - How do you perturb the data?
 - **Observe the trade-off of privacy and accuracy**
 - Effectiveness measure: Accuracy
 - Privacy level: the value of ϵ

Step 3-1: Defend against Gradient Leakage Attack, by Homomorphic Encryption (Bonus)

- Try to defend against Gradient Leakage Attack (from Step 2)
- You can apply the Homomorphic Encryption method from the below (but not limited in them)
 - [FedBoosting: Federated learning with gradient protected boosting for text recognition - ScienceDirect](#)
 - Source code: <https://github.com/Rand2AI/FedBoosting>
 - ... (The other methods based on homomorphic encryption)
- Some libraries of homomorphic encryption: <https://github.com/jonaschn/awesome-he>
- Finally, provide the results
 - How do you perturb the data?
 - **Observe the trade-off of privacy and consuming time**
 - Effectiveness measure: Accuracy
 - Time consuming

More References

- [GradAttack/papers/gradient_inversion.md](#)
- [GitHub - Koukyosyumei/AIJack: Security and Privacy Risk Simulator for Machine Learning](#)

It's okay for you choosing Step 3-1,
rather than Step 3

The grades will be depended by how many steps you completed in this final project.

Please list it in the beginning of your report.

Upload the result to Moodle

- Upload your code and the ***detailed*** reports to Moodle.
 - **PPT**
 - **Code:** You must add some comments in your code for easy understanding.
 - **Report Files:** Document (Word).
 - Including a table in your report outlining the responsibilities of each team member for this homework.
 - Zipped to a file. The file name should be “TeamName_FinalProject.zip”